文章写作Agent

- 来源: https://class.imooc.com/lesson/5285#mid=105036
- 阶段: 阶段四: 【框架应用】高效AI开发
- 周: 第9周 LangGraph：工作流编排
- 模块: 文章写作Agent
- 课节: 图文 1-1 文章写作Agent

1-1 文章写作Agent

文章写作Agent

> [!TIP] 阅读指南 上一节我们学习了 State、Node、Edge、Graph 四大核心概念。本节将通过一个完整的文章写作助手实战项目，带你体验 LangGraph 在处理复杂、有状态任务时的编排能力。建议先了解概念，再动手运行配套源码。

项目目标与流程设计

项目目标

构建一个自动写作助手,输入主题后自动生成经过润色的高质量文章。

自动写作助手

工作流设计

工作流将严格遵循以下线性流水线:

```
主题(Input) → [生成大纲] → [扩写初稿] → [润色优化] → 终稿(Output)

1

预览
```

State 设计：工作流的「草稿本」

我们需要定义一个结构化的 State 来存储写作流程中的所有中间数据:

```
from typing import TypedDict

class ArticleState(TypedDict):
    topic: str           # 文章主题(用户输入)
    outline: str         # 大纲(第1步生成)
    draft: str           # 初稿(第2步生成)
    final_article: str   # 终稿(第3步生成)
```

数据合并过程:

数据合并过程

- 初始: {"topic": "Python异步编程"}

```
{"topic": "Python异步编程"}
```

初始

- 节点1执行后: {"topic": "...", "outline": "一、什么是异步。.."} (自动合并大纲)
- 节点2执行后: {"topic": "...", "outline": "...", "draft": "完整初稿。.."} (自动合并初稿)
- 节点3执行后: {"topic": "...", "outline": "...", "draft": "...", "final_article": "成品。.."} (自动合并终稿)

节点(Node)实现

节点 1：生成大纲

任务：根据主题调用 LLM 生成文章逻辑框架。

```
from langchain_openai import ChatOpenAI

# 初始化模型 (显式设置 API Key 和 Base URL)
llm = ChatOpenAI(
    model="qwen3.6-plus",
    api_key="您的-DASHSCOPE-API-KEY",
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1",
    temperature=0.7
)

def generate_outline(state: ArticleState):
    topic = state["topic"]
    prompt = f"请为主题'{topic}'生成一个详细的文章大纲,包含3-5个要点"
    outline = llm.invoke(prompt).content

    print(f"[大纲节点] 已生成大纲,长度:{len(outline)}字")
    return {"outline": outline}
```

节点 2：扩写初稿

任务：基于大纲内容，扩写成 800-1000 字的完整初稿。

```
def write_draft(state: ArticleState):
    topic = state["topic"]
    outline = state["outline"]
    prompt = f"主题:{topic}\n大纲:\n{outline}\n\n请根据大纲扩写成完整文章。"
    draft = llm.invoke(prompt).content

    print(f"[草稿节点] 已生成初稿,长度:{len(draft)}字")
    return {"draft": draft}
```

节点 3：润色优化

任务：对初稿进行语言精炼和逻辑优化，产出最终成品。

```
def polish_article(state: ArticleState):
    draft = state["draft"]
    prompt = f"请润色以下文章,优化流畅度和逻辑性:\n\n{draft}"
    final = llm.invoke(prompt).content

    print(f"[润色节点] 已完成最终润色")
    return {"final_article": final}
```

图(Graph)构建与执行

组装工作流

我们将节点与边连接。注意这里使用了最新的 START 和 END 常量。

```
START
```

```
END
```

```
from langgraph.graph import StateGraph, START, END

# 1. 创建图
builder = StateGraph(ArticleState)

# 2. 注册节点
builder.add_node("大纲", generate_outline)
builder.add_node("草稿", write_draft)
builder.add_node("润色", polish_article)

# 3. 编排路径
builder.add_edge(START, "大纲")        # 入口
builder.add_edge("大纲", "草稿")        # 线性连接
builder.add_edge("草稿", "润色")
builder.add_edge("润色", END)           # 出口

# 4. 编译应用
app = builder.compile()
```

运行与追踪

通过 app.invoke() 触发执行，或使用 app.stream() 追踪状态流转。

```
# 执行完整流程
result = app.invoke({"topic": "AI 时代的编程范式"})
print(f"最终成品文章预览: {result['final_article'][:100]}...")

# 逐步追踪状态变化
for step in app.stream({"topic": "AI 时代的编程范式"}):
    print(step)
```

实战总结

> 完整源码路径: samples/chapter9/article_writer/article_writer.py

> 注意：全自动并非终点 虽然本节实现了一个全自动流程，但在真实的写作生产中，「LLM 生成 → 人工修改」 才是最可靠的路径。完全由 LLM 自我迭代往往会产生「幻觉累积」。 LangGraph 真正的威力在于它能极其简单地引入**「人工介入（Human-in-the-Loop）」**。我们将在本章稍后的第 6 节中专门探讨如何实现这种「人机协作」模式，让 Agent 在生成大纲后停下来等候你的审批和修改。

■ 学点英语

中文	English	音标	说明
流水线	Pipeline	/ˈpaɪplaɪn/	节点按固定顺序依次执行的线性流程模式
节点函数	Node Function	/noʊd ˈfʌŋkʃn/	接收 State 并返回更新字典的处理函数
状态累积	State Accumulation	/steɪt əˌkjuːmjəˈleɪʃn/	State 逐步合并各节点产出的数据增长模式
线性流程	Linear Flow	/ˈlɪniər floʊ/	节点按预设顺序单向执行的简单工作流模式

下节预告

现在的写作流程是线性的，一旦某个环节出错(比如大纲生成得不好),后续环节都会受到影响。在下一节中，我们将学习如何添加条件分支与循环,引入「质量检查」逻辑，让 Agent 能够在检测到质量不达标时自动「回退」并重新生成。